

MEMBANGUN SISTEM EMBEDDED DARI DASAR HINGGA REAL-TIME

Joyce Marsay Melody - 40040324630034

1. GPIO DIGITAL I/O

- GPIO = pin input/output digital
- Logika: HIGH (1), LOW (0)

- Mode:

Input: floating, pull-up, pull-down

Output: push-pull, open-drain

- Register: DDR, PORT, PIN
- Keterbatasan arus

GPIO merupakan komponen dasar dalam mikrokontroler yang berfungsi sebagai jalur komunikasi langsung dengan perangkat eksternal. Saat dikonfigurasi sebagai input, GPIO membaca kondisi tegangan dari luar, sedangkan sebagai output GPIO menghasilkan tegangan untuk mengendalikan perangkat.

Mode input seperti pull-up dan pull-down sangat penting untuk menghindari kondisi floating, yaitu keadaan di mana pin tidak memiliki referensi tegangan yang jelas sehingga dapat menghasilkan pembacaan acak. Pada mode output, push-pull digunakan untuk memberikan logika HIGH dan LOW secara aktif, sedangkan open-drain biasanya digunakan dalam komunikasi seperti I2C.

Selain itu, setiap pin memiliki batas arus tertentu sehingga tidak boleh langsung digunakan untuk beban besar tanpa rangkaian tambahan. Oleh karena itu, pemahaman GPIO menjadi dasar penting sebelum mempelajari sistem embedded yang lebih kompleks.

2. INTERRUPT & TIMER

- Interrupt = respon cepat event
- ISR (Interrupt Service Routine)
- Jenis: External dan Internal
- Timer mode: Normal dan CTC
- Prescaler

Interrupt memungkinkan mikrokontroler merespon kejadian secara langsung tanpa harus terus-menerus memeriksa kondisi (polling). Ketika interrupt terjadi, CPU menghentikan sementara proses utama dan menjalankan ISR, sehingga respon menjadi lebih cepat dan efisien. Timer bekerja berdasarkan clock internal dan digunakan untuk menghasilkan waktu yang presisi. Dengan adanya prescaler, frekuensi timer dapat dibagi sehingga cocok untuk berbagai kebutuhan waktu, seperti milidetik hingga detik. Kombinasi interrupt dan timer sangat penting dalam sistem real-time karena memungkinkan sistem menjalankan beberapa fungsi berbasis waktu tanpa mengganggu proses utama. Namun, ISR harus dibuat sesingkat mungkin agar tidak mengganggu eksekusi sistem secara keseluruhan.

3. SERIAL UART

- Asynchronous serial
- TX dan RX
- Frame data
- Baud rate
- Full duplex

UART adalah metode komunikasi serial yang sederhana dan banyak digunakan dalam sistem embedded. Karena bersifat asynchronous, komunikasi ini tidak menggunakan clock sehingga kedua perangkat harus diset dengan baud rate yang sama.

Data dikirim dalam bentuk frame yang terdiri dari start bit sebagai penanda awal, diikuti data bit, optional parity untuk error checking, dan stop bit sebagai penutup. Struktur ini memastikan data dapat diterima dengan benar oleh perangkat tujuan.

UART sering digunakan untuk komunikasi dengan komputer, modul lain, atau debugging. Meskipun sederhana, komunikasi ini memiliki keterbatasan seperti jarak yang tidak terlalu jauh dan rentan terhadap gangguan noise.

4. ADC

- Analog → Digital
- Tahapan konversi
- Resolusi
- Vref
- Error

ADC digunakan untuk mengubah sinyal analog yang bersifat kontinu menjadi data digital yang dapat diproses oleh mikrokontroler. Proses ini melibatkan sampling, quantization, dan encoding.

Resolusi ADC menentukan tingkat ketelitian hasil konversi. Semakin tinggi resolusi, semakin kecil perubahan tegangan yang dapat dideteksi. Tegangan referensi (V_{ref}) menjadi acuan utama dalam proses konversi, sehingga kestabilannya sangat penting.

Selain itu, terdapat error seperti quantization error yang tidak bisa dihindari karena proses pembulatan nilai analog ke digital. Oleh karena itu, pemilihan resolusi dan V_{ref} harus disesuaikan dengan kebutuhan aplikasi.

5. DAC & PWM

- DAC dan PWM
- Duty cycle
- Frekuensi
- Resolusi

DAC digunakan untuk menghasilkan sinyal analog dari data digital, namun karena tidak semua mikrokontroler memilikinya, PWM sering digunakan sebagai alternatif.

PWM bekerja dengan mengatur lebar pulsa sinyal digital. Tegangan rata-rata yang dihasilkan bergantung pada duty cycle, yaitu perbandingan waktu ON terhadap total periode. Semakin besar duty cycle, semakin besar tegangan rata-rata.

Frekuensi PWM juga mempengaruhi hasil, terutama pada aplikasi seperti motor dan LED. Frekuensi yang terlalu rendah dapat menyebabkan flicker atau getaran, sedangkan frekuensi yang tinggi menghasilkan output yang lebih halus.

6. I2C SENSOR

- SDA dan SCL
- Master-Slave
- Address
- ACK/NACK

I2C adalah protokol komunikasi yang efisien karena hanya menggunakan dua jalur. Mikrokontroler sebagai master mengontrol komunikasi dengan mengirimkan alamat perangkat yang dituju.

Setiap perangkat memiliki alamat unik sehingga banyak sensor dapat dihubungkan dalam satu bus. Proses komunikasi melibatkan start condition, pengiriman alamat, pertukaran data, dan stop condition.

Meskipun sederhana dan hemat pin, I2C memiliki kecepatan yang lebih rendah dibanding SPI, sehingga lebih cocok untuk komunikasi data kecil seperti sensor.

7. SPI STORAGE

- MOSI, MISO, SCK, CS
- Full duplex
- Mode SPI
- High speed

SPI adalah protokol komunikasi dengan kecepatan tinggi yang menggunakan clock sinkron. Berbeda dengan I2C, SPI tidak menggunakan addressing tetapi menggunakan pin CS untuk memilih perangkat.

Karena bersifat full duplex, data dapat dikirim dan diterima secara bersamaan. Hal ini membuat SPI sangat efisien untuk transfer data besar seperti penyimpanan atau display.

Namun, SPI membutuhkan lebih banyak pin sehingga kurang efisien untuk sistem dengan banyak perangkat.

8. DMA

- Transfer tanpa CPU
- Mode DMA
- Efisiensi tinggi

DMA memungkinkan data ditransfer langsung antara peripheral dan memori tanpa melibatkan CPU. Hal ini sangat mengurangi beban kerja CPU dan memungkinkan sistem menjalankan tugas lain secara paralel.

DMA sangat berguna dalam aplikasi dengan data besar atau kecepatan tinggi, seperti pembacaan ADC secara terus-menerus atau komunikasi data cepat. Penggunaan DMA meningkatkan performa dan efisiensi sistem secara keseluruhan.

9. FREE RTOS TASK

- Task
- Priority
- Scheduler
- State

Task dalam FreeRTOS memungkinkan sistem menjalankan beberapa proses secara bersamaan. Setiap task memiliki prioritas yang menentukan urutan eksekusi.

Scheduler bertugas mengatur eksekusi task berdasarkan prioritas dan kondisi sistem. Dengan adanya multitasking, sistem dapat menangani berbagai fungsi secara efisien tanpa harus menunggu satu proses selesai terlebih dahulu.

10. QUEUE & SEMAPHORE

- Queue (FIFO)
- Semaphore
- Mutex
- Sinkronisasi

Queue digunakan untuk mengirim data antar task secara terstruktur, sedangkan semaphore digunakan untuk mengatur akses terhadap resource agar tidak terjadi konflik.

Mutex adalah jenis semaphore khusus yang digunakan untuk melindungi resource dari akses bersamaan. Dengan mekanisme ini, sistem dapat menghindari masalah seperti race condition dan deadlock.

11. TIMER & NOTIFICATION

- Software timer
- Task notification
- Efisien

Software timer memungkinkan eksekusi fungsi secara berkala tanpa menggunakan hardware timer secara langsung. Hal ini mempermudah pengelolaan waktu dalam sistem multitasking.

Task notification adalah metode komunikasi antar task yang lebih ringan dibandingkan queue karena tidak memerlukan buffer data. Ini membuat sistem lebih cepat dan efisien.

12. MEMORY MANAGEMENT

- Stack & Heap
- Fragmentasi
- Overflow
- Strategi

Manajemen memori sangat penting dalam sistem embedded karena keterbatasan kapasitas. Stack digunakan untuk task, sedangkan heap untuk alokasi dinamis.

Masalah seperti fragmentasi dan overflow dapat menyebabkan sistem tidak stabil atau crash. Oleh karena itu, diperlukan strategi seperti penggunaan alokasi statis dan monitoring memori secara berkala.

13. NETWORK

- WiFi / Ethernet
- TCP / IP
- HTTP, MQTT
- IoT

Network memungkinkan sistem embedded terhubung dengan perangkat lain atau internet. Dengan protokol seperti TCP/IP, data dapat dikirim secara andal, sedangkan MQTT digunakan untuk komunikasi ringan pada IoT.

Dengan adanya konektivitas ini, sistem dapat melakukan monitoring dan kontrol secara jarak jauh, yang menjadi dasar dari teknologi IoT modern.

14. INDUSTRIAL APPLICATION

- PLC
- SCADA
- Real-time
- Safety

Dalam industri, sistem **embedded** digunakan untuk mengontrol dan memonitor proses produksi secara otomatis. PLC digunakan untuk kontrol logika, sedangkan SCADA digunakan untuk monitoring secara terpusat.

Sistem ini harus memiliki keandalan tinggi dan mampu bekerja secara real-time karena digunakan dalam lingkungan kritis. Oleh karena itu, desain sistem harus memperhatikan faktor keamanan dan kestabilan.

KESIMPULAN

Secara keseluruhan, sistem embedded merupakan integrasi antara perangkat keras dan lunak yang dirancang untuk fungsi tertentu. Dengan adanya komunikasi, manajemen memori, dan RTOS, sistem menjadi lebih kompleks namun juga lebih powerful. Teknologi ini sangat penting dalam perkembangan IoT dan industri modern.



THANK YOU